

Arduino 실습

안휘준

코드 분석

탁구 게임은 버튼 입력을 읽고, 패들 위치와 공 위치를 바꾼 뒤 OLED 화면에 다시 그리는 구조였다. 공이 벽이나 패들에 닿으면 이동 방향을 바꾸고, 공을 놓치면 Game Over가 된다.

미니 오실로스코프는 외부 전압을 ADC로 숫자로 바꾸고, 그 값을 여러 번 저장한 뒤 OLED에 파형으로 그린다. 여기서 ADC는 아날로그 전압을 Arduino가 계산할 수 있는 숫자로 바꾸는 장치이다.

C++와 Verilog의 구조 차이

C++에서는 Arduino의 CPU가 코드를 위에서 아래로 순서대로 실행한다. 예를 들어 버튼 읽기 → 공 위치 계산 → 충돌 확인 → 화면 출력 순서로 처리한다.

Verilog는 프로그램 명령을 쓰는 느낌보다 회로의 구조를 설명하는 언어에 가깝다. 공 위치 계산 회로, 충돌 확인 회로, 화면 출력 회로처럼 여러 기능을 따로 만들 수 있고, 이 회로들은 동시에 동작할 수 있다.

Compiling

C++ 소스코드는 사람이 읽기 쉽게 작성한 코드이다. 하지만 ATmega328P 같은 마이크로컨트롤러는 이 코드를 그대로 이해하지 못한다. 그래서 compile을 해서 CPU가 실행할 수 있는 기계어 형태로 바꾼다. 쉽게 말하면 compiler는 C++ 코드를 칩이 실행할 수 있는 형태로 번역해 주는 도구라고 이해했다. Compiling은 Arduino CLI를 이용해 C++ 코드를 compile했다.

합성(Synthesis)

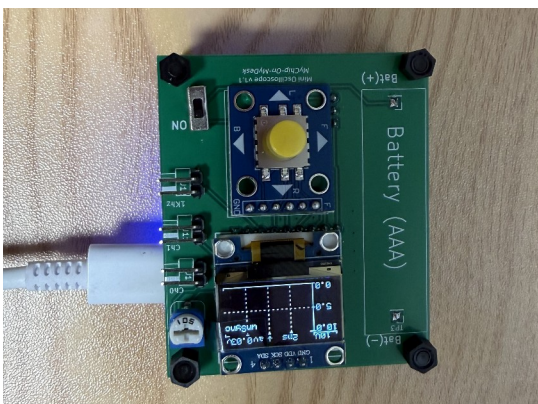
Verilog도 작성한 코드가 바로 실제 회로가 되는 것은 아니다. 합성이라는 과정을 거쳐 Verilog 내용을 논리게이트, 플립플롭 같은 실제 디지털 회로 구조로 바꾼다.

정리하면 C++의 compile은 실행할 명령어를 만드는 과정이고, Verilog의 synthesis는 만들 회로의 구조를 만드는 과정이라고 이해했다.

실행 결과

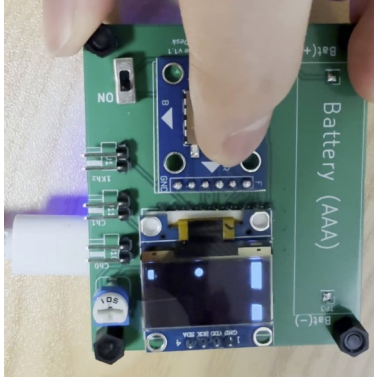
미니 오실로스코프는 보드에 업로드한 뒤 정상적으로 실행되었고, 보드의 약 1 kHz 테스트 신호를 측정했을 때 약 970~980 Hz가 표시되는 것을 확인했다.

탁구 게임도 컴파일과 업로드 후 OLED에서 정상적으로 실행되는 것을 확인했다.



원) 아두이노 보드 pc 연결 직후

오) 미니 오실로스코프 실행 결과



탁구 게임기 실행 결과

느낀점

처음에는 C++ 코드와 Verilog 코드가 비슷한 방식으로 동작한다고 생각했지만, 실습하면서 차이가 조금 보였다. C++는 하나의 CPU가 명령을 순서대로 실행하는 방식이고, Verilog는 여러 회로가 동시에 동작하도록 구조를 만드는 방식이라는 점이 가장 큰 차이라고 느꼈다.

또 C++에서는 compile이 필요하고, Verilog에서는 synthesis가 필요하다는 점도 알게 되었다. 둘 다 작성한 코드를 그대로 사용하는 것이 아니라, 실제 칩이나 회로가 사용할 수 있는 형태로 바꾸는 과정이라는 점이 흥미로웠다.